KOLEJ PROFESIONAL MARA INDERA MAHKOTA

DIPLOMA IN COMPUTER SCEINCE

COURSE NAME	:	DATABASE FUNDAMENTALS
COURSE CODE	:	CSC 1393
SESSION	:	3 2025/2026
TYPE OF ASSESSMENT	:	FINAL PROJECT
DURATION	:	3 WEEKS

CLO 3: Construct a database project and technical documentation based on an existing database design. (P3, PLO6)

INSTRUCTION TO CANDIDATES:

1. Carefully check the submission date and the instructions given with the assignment/project. Late submission will not be accepted.
2. If you use other people's work or ideas in your assignment/project, kindly reference them using APA system.

Personal Details	
Name	MUHAMMAD IRFAN MUHAIMIN BIN MUHD ZAMIZI
I/D Number	ICS25-02-005
Class	DCS2A
Lecturer	MADAM HAFIZAH

Section / Question No.	**Marks**
1	
2	
Total	**/ 50**

We hereby confirmed that this assignment is our own work and not copied or plagiarized from any source. We have referenced the sources from which the information is obtained by us for this assignment.

Signature_____ Date 19/02/2026_____

SCENARIO

In the dynamic world of hospitality and tourism, delivering exceptional guest experiences while managing operational efficiency is the key to success. A comprehensive database system enables hotels and resorts to streamline reservations, personalize guest services, optimize resource allocation, and build lasting customer loyalty through seamless service integration.

Azure Sands Resort & Spa, a premier beachfront destination, is undertaking a digital transformation to upgrade its legacy systems into a unified Resort Management Platform. As the lead Database Developer, you are assigned to design, develop, and implement a relational database for a specific module of this new system.

Choose **only one (1)** of the following modules to develop:

1. Online Room Reservation & Booking Engine
2. Guest Check-in/Check-out & Profile Management
3. Housekeeping Staff Assignment & Room Status
4. Restaurant & Dining Reservation Management
5. Spa & Wellness Activity Booking
6. Conference & Event Hall Booking
7. Inventory Management for Minibar & Amenities
8. Maintenance Request & Work Order Tracking
9. Guest Feedback & Review Management System
10. Loyalty Program & Membership Tier Management
11. Tour & Excursion Booking Desk
12. Staff Roster & Duty Scheduling
13. Food & Beverage (F&B) Cost Control & Menu Engineering
14. Lost & Found Item Logging
15. Valet Parking & Vehicle Management
16. Wedding & Special Event Package Coordination
17. Supplier & Vendor Management for Resort Operations
18. Energy & Utilities Consumption Monitoring
19. Water Sports Equipment Rental & Safety Log
20. Kids' Club Activity Registration & Attendance
21. Corporate & Travel Agent Commission Tracking

22. VIP & Celebrity Guest Protocol Management
23. Noise Complaint & Neighbour Disturbance Logging
24. Real-time Shuttle Bus Schedule & Guest Pick-up Management
25. Staff Training & Certification Renewal Tracking

For the selected module, you must design and populate **at least four (4)** normalized database tables with sufficient sample records to support the execution of appropriate and meaningful SQL queries.

TASK

For the Final Project, you need to complete all the following tasks:

1.0 Database Implementation

Using SQL (Structured Query Language), **construct (P3)** a relational database for the selected module using an appropriate Database Management System (DBMS).

1.1 SQL Syntax
- Adhere to standard SQL syntax conventions throughout your implementation.

1.2 Data Definition Language (DDL)
- Create **at least four (4)** normalized tables for the chosen module.

1.3 Data Manipulation Language (DML)
- Populate each table with **5–10** realistic sample records relevant to the resort's operations.
- Perform **two (2)** update operations:
  a) Update a record using its primary key.
  b) Update a record using a foreign key relationship.

1.4 Queries and Joins
- Develop **four (4)** meaningful SQL queries:
  a) **Two (2)** basic queries retrieving data from a single table.
  b) **Two (2)** advanced queries using table joins.

1.5 DBMS Implementation
- Execute all SQL statements (DDL, DML, and queries) in a DBMS of your choice.

2.0 Database Technical Documentation

Prepare comprehensive technical documentation for your database project, organized into the following sections:

2.1 Database Design Documentation
2.2 SQL Documentation
2.3 Database Administration Documentation

Guideline for Technical Documentation:

1.0 Overview
2.0 Database Design Documentation
    2.1 ERD
    2.1 Normalization
3.0 SQL Documentation
    3.1 Data Dictionary
    3.2 SQL codes (Task 1)
4.0 Database Administration Documentation
    4.1 Security
    4.2 Backup and Restore
    4.3 Basic Maintenance
5.0 References
6.0 Appendices

Rubrics for Database Implementation

No	Sub attributes	Task	Limited (1 mark)	Satisfactory (2 marks)	Excellent (3 marks)	Marks Weight	Marks Obtained
1	**Reproduce and process information**	**1.0 Using SQL (Structured Query Language), construct a database with suitable table structure using appropriate DBMS**					
		1.1 SQL Syntax	**Syntax Accuracy & SQL Conventions**				
			❑ SQL syntax has significant errors, leading to incorrect or unexpected results ❑ Rarely use proper SQL conventions	❑ SQL syntax is mostly accurate, with minor errors that do not affect functionality ❑ Use some SQL conventions but inconsistently	❑ SQL syntax is entirely accurate with no errors. ❑ Consistently uses proper SQL conventions, including capitalization, indentation, and comments	1	**/3**
		1.2 Data Definition Language (DDL)	❑ DDL statements are incomplete with significant errors	❑ DDL statements are mostly accurate with minor issues. (e.g., inappropriate data types)	❑ DDL statements for table creation are accurate, including appropriate data types, primary keys, and foreign keys. ❑ Appropriate constraints (e.g., PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, are applied effectively.	3	**/9**
		1.3 Data Manipulation Language (DML)	**Data Insertion**				
			❑ INSERT statements are incomplete with significant error ❑ Insufficient records populate in tables created ❑ Impropriate relation name for each table	❑ INSERT statements are mostly accurate with minor issues. ❑ Sufficient records populate in tables created ❑ Impropriate relation name for each table	❑ INSERT statements are accurate, inserting all required data correctly. ❑ Sufficient records populate in tables created ❑ Appropriate relation name for each table	2	**/6**
			Data update				
			❑ UPDATE statements are incomplete with significant errors.	❑ UPDATE statements are mostly accurate with minor issues	❑ All UPDATE statements are accurate, correctly updating the required data.	2	**/6**

		1.4 Queries and Joins	**Basic Queries**				
			❑ Basic SELECT queries are incomplete with significant errors. ❑ Student is unable to clearly explain the query or its purpose, and the relevance to the project scenario is unclear or incorrect.	❑ Basic SELECT queries are mostly accurate with minor issues. ❑ Gives a basic explanation of the query and mentions its purpose, but the explanation lacks detail or clear connection to the project scenario.	❑ Basic SELECT queries are accurate, retrieving the correct data as specified ❑ Clearly explains what the query does and why it is important. ❑ The query is relevant to the project scenario and supports system or user needs.	1	**/3**
			Advanced Queries				
			❑ JOIN operations are incomplete with significant errors. ❑ Student is unable to clearly explain the query or its purpose, and the relevance to the project scenario is unclear or incorrect.	❑ JOIN operations are mostly accurate with minor issues ❑ Gives a basic explanation of the query and mentions its purpose, but the explanation lacks detail or clear connection to the project scenario.	❑ JOIN operations (example: INNER, LEFT, RIGHT, FULL) are accurate and efficiently implemented. ❑ Clearly explains what the query does and why it is important. ❑ The query is relevant to the project scenario and supports system or user needs.	3	**/9**
		1.5 DBMS Implementation	**1 Mark**	**2 Marks**		1	**/2**
			❑ Chooses a suitable DBMS (e.g., MySQL, PostgreSQL, SQLite) but minor inefficiency used of DBMS. ❑ Database implementation with DBMS that is partially successful	❑ Chooses a suitable DBMS (e.g., MySQL, PostgreSQL, SQLite) ❑ The database is demonstrating effective use of aDBMS ❑ Successfully implements a database using DBMS.			
						Database Implementation Mark	**/38**

Rubrics for Database Technical Documentation

No	Sub attributes	Task	Limited (0.5 mark)	Satisfactory (1 mark)	Excellent (1.5 marks)	Marks Weight	Marks Obtained
2	**Design**	**2.1 Database Design Documentation**					
		2.1.1 ERD	❑ The ERD is incomplete or contains significant errors, such as missing entities, attributes, relationships, or unclear cardinality.	❑ The ERD includes most entities, attributes, and relationships, but some labels, cardinality, or connections may be unclear or missing minor details.	❑ The ERD is complete, accurate, and well-organized. ❑ All entities, attributes, and relationships are clearly labeled, and appropriate cardinality is shown.	2	**/3**
		2.1.2 Normalization	❑ The schema is poorly normalized or not normalized at all, with multiple violations of 1NF, 2NF, or 3NF ❑ The 3NF schema did not match with the ERD provided.	❑ The schema is partially normalized but has minor errors or omissions in 1NF, 2NF, or 3NF ❑ The 3NF schema match with the ERD provided.	❑ The schema is fully normalized through 1NF, 2NF, and 3NF. ❑ No transitive dependencies remain. ❑ The 3NF schema match with the ERD provided.	1	**/1.5**
	Reproduce and process information	**2.2 SQL Documentation**					
		2.2.1 Data Dictionary	❑ The data dictionary is incomplete or disorganized. Key details such as attribute names, data types, or constraints are missing, incorrect, or insufficiently detailed. ❑ The Data Dictionary did not match with the ERD provided.	❑ The data dictionary includes most required details but may lack clarity or precision in some areas. ❑ The Data Dictionary match with the ERD provided.	❑ The data dictionary is complete and well-organized. ❑ It includes all required details such as table name, attribute names, data types, sizes, PK/FK, constraints, and descriptions. All information is accurate and clearly presented. ❑ The Data Dictionary match with the ERD provided.	2	**/3**
		2.2.2 SQL Codes	**Task 1.0**				

		2.3 Database Administration Documentation					
		2.3.1 Security	❑ Security details are incomplete, disorganized, or incorrect. ❑ Critical information such as host, port, or roles is missing or poorly defined.	❑ Security details are mostly complete but may lack precision. Minor gaps in clarity or relevance.	❑ Security details are complete and accurate. Includes database host, port, usernames and roles. ❑ All roles are clearly defined and relevant.	1	**/1.5**
		2.3.2 Backup and Restore	❑ Backup and restore details are incomplete or inaccurate. ❑ Key information such as backup policy, location, or commands is missing or unclear.	❑ Backup and restore details are mostly complete but may lack clarity or minor details	❑ Backup and restore details are complete and accurate. ❑ Includes backup policy, backup location, and clear, executable commands for backup and restore.	1	**/1.5**
		2.3.3 Basic Maintenance	❑ Maintenance details are incomplete, disorganized, or inaccurate. ❑ Commands and frequency for optimizing and checking tables are missing or unclear.	❑ Maintenance details are mostly complete but may lack clarity or depth	❑ Maintenance details are complete and accurate. ❑ Includes executable commands and frequency for optimizing and checking tables.	1	**/1.5**
						Database Technical Documentation Mark	**/12**
						Total Marks Earned (Database Implementation + Database Technical Documentation)	**/50**
						Total Percentage (40%)	

ACKNOWLEDGEMENT

Assalamualaikum warahmatullah. First and foremost, I would like to express my sincere gratitude to all the individuals who have supported me in completing this assignment. My lecturer, Madam Hafizah, for her continuous guidance, feedback, and valuable insights throughout the making of this project. To my family for their constant encouragement, patience and understanding that have been pillars of my success. I would also like to express gratitude to my friends who contributed their ideas and perspectives that enriched the assignment. Thank you to everyone for shaping this assignment and enhancing our learning experience.

OVERVIEW

Azure Sands Resort & Spa is undertaking a digital transformation to upgrade its legacy systems. This project focuses on the Wedding & Event Coordination module. As the lead Database Developer, a relational database is designed to manage venue bookings, customer information, available packages, and booking transactions for the resort.

One customer can make many bookings, but each booking belongs to only one customer. Each booking is associated with one venue and one package. A venue can be included in many bookings, and a package can also be selected for many bookings. The database stores all relevant information including event type, event date, guest count, total amount, balance due, and booking status.

The system consists of four (4) normalized tables: CUSTOMER, VENUE, PACKAGE, and BOOKING.

DATABASE DESIGN DOCUMENTATION

3.1 ERD

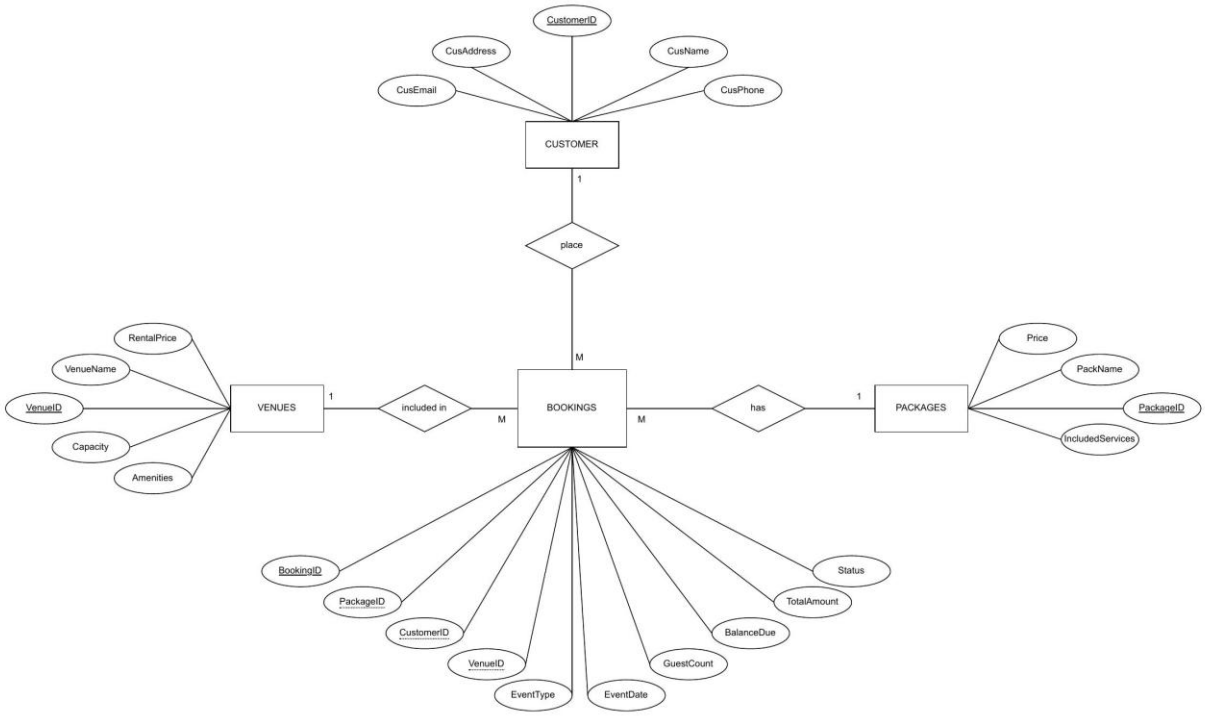

Figure 1: Entity Relationship Diagram (ERD) – Event Booking System

3.2 NORMALIZATION

0NF

BOOKINGS

BookingID	CustomerID	CusName	CusEmail	CusPhone	VenueID	VenueName	PackageID	PackName	EventType	EventDate	GuestCount	TotalAmount	BalanceDue	Status
1	1000	Ahmad Hafiz	ahmad.hafiz@email.com	013-4567890	2000	Grand Ballroom	3002	Premium	Wedding	2026-03-06	350	11000	3000	Confirmed
2	1001	Siti Nurhaliza	siti.nur@email.com	019-2345678	2002	Executive Hall	3003	Corporate	Corporate Meeting	2026-02-19	80	5000	0	Paid

In the BOOKINGS table (0NF), the table contains repeating groups and non-atomic values. Customer details such as name, email, phone are repeated across rows. Venue and package information are also repeated, violating the first normal form.

1NF

BOOKINGS (BookingID, CustomerID, CusName, CusEmail, CusPhone, VenueID, VenueName, Capacity, RentalPrice, Amenities, PackageID, PackName, Price, IncludedServices, EventType, EventDate, GuestCount, TotalAmount, BalanceDue, Status)

To achieve 1NF, all non-atomic values are separated into individual columns. Each row now contains only atomic values. Repeating groups are removed. The highlighted attributes (CusName, VenueName, PackName) remain as they still contain partial dependencies.

BookingID	CustomerID	CusName	CusPhone	VenueID	VenueName	PackageID	PackName	EventType	EventDate	GuestCount	TotalAmount	BalanceDue	Status
1	1000	Ahmad Hafiz	013-4567890	2000	Grand Ballroom	3002	Premium Pkg	Wedding	2026-03-06	350	11000.00	3000.00	Confirmed
2	1001	Siti Nurhaliza	019-2345678	2002	Executive Hall	3003	Corporate Pkg	Corp. Meeting	2026-02-19	80	5000.00	0.00	Paid

2NF

In 2NF, partial dependencies are removed from the 1NF table. All non-key attributes must be fully functionally dependent on the entire primary key.

BOOKING (BookingID, CustomerID, VenueID, PackageID, EventType, EventDate, GuestCount, TotalAmount, BalanceDue, Status)

CUSTOMER (CustomerID, CusName, CusEmail, CusPhone, CusAddress)

VENUE (VenueID, VenueName, Capacity, RentalPrice, Amenities)

PACKAGE (PackageID, PackName, Price, IncludedServices)

Attributes such as CusName, CusEmail, CusPhone depend only on CustomerID, not on the full composite key. Similarly, VenueName depends only on VenueID and PackName depends only on PackageID. These partial dependencies are resolved by moving them to separate tables.

3NF

In 3NF, transitive dependencies are removed. All non-key attributes must depend directly on the primary key and not on any other non-key attribute.

BOOKING

BookingID	CustomerID	VenueID	PackageID	EventType	Event Date	GuestCount	TotalAmount	Balance Due	Status
1	1000	2000	3002	Wedding	2026-03-06	350	11000.00	3000.00	Confirmed
2	1001	2002	3003	Corporate Meeting	2026-02-19	80	5000.00	0.00	Paid
3	1002	2001	3000	Birthday Party	2026-03-21	120	4300.00	2150.00	Confirmed
4	1003	2000	3004	Wedding	2026-04-05	450	13000.00	6500.00	Pending
5	1004	2003	3001	Anniversary	2026-02-24	75	6700.00	0.00	Paid
6	1000	2004	3003	Training Workshop	2026-02-14	40	4300.00	1000.00	Confirmed
7	1002	2002	3001	Product Launch	2026-03-11	100	6000.00	3000.00	Confirmed

CUSTOMER

CustomerID	CusName	CusEmail	CusPhone	CusAddress
1000	Ahmad Hafiz	ahmad.hafiz@email.com	013-4567890	Jalan Tun Razak, Kuantan
1001	Siti Nurhaliza	siti.nur@email.com	019-2345678	Bandar Indera Mahkota, Kuantan
1002	Elizabeth Tan	lizzytan@email.com	012-7654321	Inderapura, Kuantan
1003	Mohd Azwan Md Nor	wakdoyok@email.com	011-8765432	Jalan IM3, Kuantan
1004	Mohd Syafiq Shamim	syafiqkyle@email.com	014-3456789	Kotasas, Kuantan

VENUE

VenueID	VenueName	Capacity	RentalPrice	Amenities
2000	Grand Ballroom	500	3500.00	AC, Sound System, Projector, Stage
2001	Garden Pavilion	200	1800.00	Outdoor, Lighting, Gazebo

2002	Executive Hall	150	1500.00	AC, WiFi, Projector, Whiteboard
2003	Rooftop Terrace	100	2200.00	Outdoor, City View, Bar Counter
2004	Conference Room A	50	800.00	AC, Video Conference, Flipchart

PACKAGE

PackageID	PackName	Price	IncludedServices
3000	Basic Package	2500.00	Basic Catering, Standard Decoration
3001	Normal Package	4500.00	Full Catering, Premium Decoration, Photographer
3002	Premium Package	7500.00	Luxury Catering, Designer Decoration, Photography & Videography, MC
3003	Corporate Package	3500.00	Meeting Catering, AV Equipment, Technical Support
3004	Wedding Package	9500.00	Premium Catering, Wedding Decoration, Photography & Videography, MC, Bridal Room

The 3NF tables have no transitive dependencies. All non-key attributes in each table depend directly and only on the primary key of that table. The final 3NF schema matches the ERD provided.

SQL DOCUMENTATION

4.1 Data Dictionary

Table	Field	Data Type	Size	PK/FK	Nullable	Description
CUSTOMER	**CustomerID**	INT	10	PK	NO	Unique ID for each customer
	CusName	VARCHAR	100	-	NO	Full name of the customer
	CusEmail	VARCHAR	100	-	NO	Email address of the customer
	CusPhone	VARCHAR	15	-	NO	Phone number of the customer
	CusAddress	TEXT	-	-	YES	Residential address of the customer

Table	Field	Data Type	Size	PK/FK	Nullable	Description
VENUE	**VenueID**	INT	10	PK	NO	Unique ID for each venue
	VenueName	VARCHAR	100	-	NO	Name of the venue
	Capacity	INT	-	-	NO	Maximum number of guests the venue can accommodate
	RentalPrice	DECIMAL	10,2	-	NO	Price to rent the venue
	Amenities	TEXT	-	-	YES	List of facilities available at the venue

Table	Field	Data Type	Size	PK/FK	Nullable	Description
PACKAGE	**PackageID**	INT	10	PK	NO	Unique ID for each package
	PackName	VARCHAR	100	-	NO	Name of the package
	Price	DECIMAL	10,2	-	NO	Price of the package
	IncludedServices	TEXT	-	-	YES	Services included in the package

Table	Field	Data Type	Size	PK/FK	Nullable	Description
BOOKING	**BookingID**	INT	10	PK	NO	Unique ID for each booking
	CustomerID	INT	10	FK	NO	Refers to CUSTOMER(CustomerID)
	VenueID	INT	10	FK	NO	Refers to VENUE(VenueID)
	PackageID	INT	10	FK	NO	Refers to PACKAGE(PackageID)
	EventType	VARCHAR	50	-	NO	Type of event (e.g., Wedding, Corporate Meeting)
	EventDate	DATE	-	-	NO	Date of the event
	GuestCount	INT	-	-	NO	Number of guests expected
	TotalAmount	DECIMAL	10,2	-	NO	Total booking amount (venue + package)
	BalanceDue	DECIMAL	10,2	-	YES	Remaining balance to be paid
	Status	ENUM	-	-	NO	Booking status: Pending / Confirmed / Paid

4.2 SQL Codes

4.2.1 SQL Syntax for Data Definition Language (DDL)

1) Creating Table CUSTOMER

```
MariaDB [event]> CREATE TABLE CUSTOMER (
    -> CustomerID INT(10) PRIMARY KEY,
    -> CusName VARCHAR(100) NOT NULL,
    -> CusEmail VARCHAR(100) NOT NULL,
    -> CusPhone VARCHAR(15) NOT NULL,
    -> CusAddress TEXT
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Figure 1: Commands to create CUSTOMER table

```
MariaDB [event]> DESCRIBE CUSTOMER;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| CustomerID  | int(10)      | NO   | PRI | NULL    |       |
| CusName     | varchar(100) | NO   |     | NULL    |       |
| CusEmail    | varchar(100) | NO   |     | NULL    |       |
| CusPhone    | varchar(15)  | NO   |     | NULL    |       |
| CusAddress  | text         | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
```

Figure 2: CUSTOMER table structure

2) Creating Table VENUE

```
MariaDB [event]> CREATE TABLE VENUE (
    -> VenueID INT(10) PRIMARY KEY,
    -> VenueName VARCHAR(100) NOT NULL,
    -> Capacity INT NOT NULL,
    -> RentalPrice DECIMAL(10,2) NOT NULL,
    -> Amenities TEXT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Figure 3: Commands to create VENUE table

```
MariaDB [event]> DESCRIBE VENUE;
+-------------+--------------+------+-----+---------+-------+
| Field       | Type         | Null | Key | Default | Extra |
+-------------+--------------+------+-----+---------+-------+
| VenueID     | int(10)      | NO   | PRI | NULL    |       |
| VenueName   | varchar(100) | NO   |     | NULL    |       |
| Capacity    | int(11)      | NO   |     | NULL    |       |
| RentalPrice | decimal(10,2)| NO   |     | NULL    |       |
| Amenities   | text         | YES  |     | NULL    |       |
+-------------+--------------+------+-----+---------+-------+
```

Figure 4: VENUE table structure

3) Creating Table PACKAGE

```
MariaDB [event]> CREATE TABLE PACKAGE (
    -> PackageID INT(10) PRIMARY KEY,
    -> PackName VARCHAR(100) NOT NULL,
    -> Price DECIMAL(10,2) NOT NULL,
    -> IncludedServices TEXT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

Figure 5: Commands to create PACKAGE table

```
MariaDB [event]> DESCRIBE PACKAGE;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| PackageID        | int(10)      | NO   | PRI | NULL    |       |
| PackName         | varchar(100) | NO   |     | NULL    |       |
| Price            | decimal(10,2)| NO   |     | NULL    |       |
| IncludedServices | text         | YES  |     | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
```

Figure 6: PACKAGE table structure

4) Creating Table BOOKING

```
MariaDB [event]> CREATE TABLE BOOKING (
    -> BookingID INT(10) PRIMARY KEY,
    -> CustomerID INT(10),
    -> VenueID INT(10),
    -> PackageID INT(10),
    -> EventType VARCHAR(50) NOT NULL,
    -> EventDate DATE NOT NULL,
    -> GuestCount INT NOT NULL,
    -> TotalAmount DECIMAL(10,2) NOT NULL,
    -> BalanceDue DECIMAL(10,2),
    -> Status ENUM('Pending','Confirmed','Paid') DEFAULT 'Pending',
    -> FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
    -> FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID),
    -> FOREIGN KEY (PackageID) REFERENCES PACKAGE(PackageID)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Figure 7: Commands to create BOOKING table

```
MariaDB [event]> DESCRIBE BOOKING;
+------------+-----------------------------------+------+-----+---------+------
-+
| Field      | Type                              | Null | Key | Default | Extra
|
+------------+-----------------------------------+------+-----+---------+------
-+
| BookingID  | int(10)                           | NO   | PRI | NULL    |
|
| CustomerID | int(10)                           | YES  | MUL | NULL    |
|
| VenueID    | int(10)                           | YES  | MUL | NULL    |
|
| PackageID  | int(10)                           | YES  | MUL | NULL    |
|
| EventType  | varchar(50)                       | NO   |     | NULL    |
|
| EventDate  | date                              | NO   |     | NULL    |
|
| GuestCount | int(11)                           | NO   |     | NULL    |
|
| TotalAmount| decimal(10,2)                     | NO   |     | NULL    |
|
| BalanceDue | decimal(10,2)                     | YES  |     | NULL    |
|
| Status     | enum('Pending','Confirmed','Paid') | YES  |     | Pending |
|
+------------+-----------------------------------+------+-----+---------+------
-+
```

Figure 8: BOOKING table structure

4.2.2 SQL Syntax for Data Manipulation Language (DML)

4.2.2.1 Insert Record

1) CUSTOMER Table

```
MariaDB [event]> INSERT INTO CUSTOMER (CustomerID, CusName, CusEmail, CusPhone,
CusAddress)
    -> VALUES
    -> (1000, 'Ahmad Hafiz', 'ahmad.hafiz@email.com', '013-4567890', 'Jalan Tun
Razak, Kuantan'),
    -> (1001, 'Siti Nurhaliza', 'siti.nur@email.com', '019-2345678', 'Bandar
Indera Mahkota, Kuantan'),
    -> (1002, 'Elizabeth Tan', 'lizzytan@email.com', '012-7654321', 'Inderapura,
Kuantan'),
    -> (1003, 'Mohd Azwan Md Nor', 'wakdoyok@email.com', '011-8765432', 'Jalan
IM3, Kuantan'),
    -> (1004, 'Mohd Syafiq Shamim', 'syafiqkyle@email.com', '014-3456789',
'Kotasas, Kuantan');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Figure 1: Commands to insert CUSTOMER table's records

```
MariaDB [event]> SELECT * FROM CUSTOMER;
+------------+--------------------+--------------------------+-------------+----
----------------------------+
| CustomerID | CusName            | CusEmail                 | CusPhone    |
CusAddress                    |
+------------+--------------------+--------------------------+-------------+----
----------------------------+
|       1000 | Ahmad Hafiz        | ahmad.hafiz@email.com    | 013-4567890 |
Jalan Tun Razak, Kuantan       |
|       1001 | Siti Nurhaliza     | siti.nur@email.com       | 019-2345678 |
Bandar Indera Mahkota, Kuantan |
|       1002 | Elizabeth Tan      | lizzytan@email.com       | 012-7654321 |
Inderapura, Kuantan            |
|       1003 | Mohd Azwan Md Nor  | wakdoyok@email.com       | 011-8765432 |
Jalan IM3, Kuantan             |
|       1004 | Mohd Syafiq Shamim | syafiqkyle@email.com     | 014-3456789 |
Kotasas, Kuantan               |
+------------+--------------------+--------------------------+-------------+----
----------------------------+
5 rows in set (0.00 sec)
```

Figure 2: CUSTOMER table with records

2) VENUE Table

```
MariaDB [event]> INSERT INTO VENUE (VenueID, VenueName, Capacity, RentalPrice,
Amenities)
    -> VALUES
    -> (2000, 'Grand Ballroom', 500, 3500.00, 'AC, Sound System, Projector,
Stage'),
    -> (2001, 'Garden Pavilion', 200, 1800.00, 'Outdoor, Lighting, Gazebo'),
    -> (2002, 'Executive Hall', 150, 1500.00, 'AC, WiFi, Projector,
Whiteboard'),
    -> (2003, 'Rooftop Terrace', 100, 2200.00, 'Outdoor, City View, Bar
Counter'),
    -> (2004, 'Conference Room A', 50, 800.00, 'AC, Video Conference,
Flipchart');
Query OK, 5 rows affected (0.01 sec)
```

Figure 3: Commands to insert VENUE table's records

```
MariaDB [event]> SELECT * FROM VENUE;
+----------+------------------+----------+-------------+-----------------------
----------+
| VenueID | VenueName         | Capacity | RentalPrice | Amenities
|
+----------+------------------+----------+-------------+-----------------------
----------+
|    2000 | Grand Ballroom    |      500 |     3500.00 | AC, Sound System,
Projector, Stage|
|    2001 | Garden Pavilion   |      200 |     1800.00 | Outdoor, Lighting,
Gazebo        |
|    2002 | Executive Hall    |      150 |     1500.00 | AC, WiFi, Projector,
Whiteboard  |
|    2003 | Rooftop Terrace   |      100 |     2200.00 | Outdoor, City View, Bar
Counter  |
|    2004 | Conference Room A |       50 |      800.00 | AC, Video Conference,
Flipchart  |
+----------+------------------+----------+-------------+-----------------------
----------+
```

Figure 4: VENUE table with records

3) PACKAGE Table

```
MariaDB [event]> INSERT INTO PACKAGE (PackageID, PackName, Price,
IncludedServices)
    -> VALUES
    -> (3000, 'Basic Package', 2500.00, 'Basic Catering, Standard Decoration'),
    -> (3001, 'Normal Package', 4500.00, 'Full Catering, Premium Decoration,
Photographer'),
    -> (3002, 'Premium Package', 7500.00, 'Luxury Catering, Designer Decoration,
Photography & Videography, MC'),
    -> (3003, 'Corporate Package', 3500.00, 'Meeting Catering, AV Equipment,
Technical Support'),
    -> (3004, 'Wedding Package', 9500.00, 'Premium Catering, Wedding Decoration,
Photography & Videography, MC, Bridal Room');
Query OK, 5 rows affected (0.01 sec)
```

Figure 5: Commands to insert PACKAGE table's records

```
MariaDB [event]> SELECT * FROM PACKAGE;
+-----------+-----------------+---------+-------------------------------------
-----------------+
| PackageID | PackName         | Price   | IncludedServices
|
+-----------+-----------------+---------+-------------------------------------
-----------------+
|      3000 | Basic Package    | 2500.00 | Basic Catering, Standard Decoration
|
|      3001 | Normal Package   | 4500.00 | Full Catering, Premium Decoration,
Photographer       |
|      3002 | Premium Package  | 7500.00 | Luxury Catering, Designer Decoration,
Phot. & Video.  |
|      3003 | Corporate Package| 3500.00 | Meeting Catering, AV Equipment,
Technical Support     |
|      3004 | Wedding Package  | 9500.00 | Premium Catering, Wedding Decoration,
Phot. & Video.  |
+-----------+-----------------+---------+-------------------------------------
-----------------+
```

Figure 6: PACKAGE table with records

4) BOOKING Table

```
MariaDB [event]> INSERT INTO BOOKING
    -> (BookingID, CustomerID, VenueID, PackageID, EventType, EventDate,
GuestCount, TotalAmount, BalanceDue, Status)
    -> VALUES
    -> (1, 1000, 2000, 3002, 'Wedding', '2026-03-06', 350, 11000.00, 3000.00,
'Confirmed'),
    -> (2, 1001, 2002, 3003, 'Corporate Meeting', '2026-02-19', 80, 5000.00,
0.00, 'Paid'),
    -> (3, 1002, 2001, 3000, 'Birthday Party', '2026-03-21', 120, 4300.00,
2150.00, 'Confirmed'),
    -> (4, 1003, 2000, 3004, 'Wedding', '2026-04-05', 450, 13000.00, 6500.00,
'Pending'),
    -> (5, 1004, 2003, 3001, 'Anniversary', '2026-02-24', 75, 6700.00, 0.00,
'Paid'),
    -> (6, 1000, 2004, 3003, 'Training Workshop', '2026-02-14', 40, 4300.00,
1000.00, 'Confirmed'),
    -> (7, 1002, 2002, 3001, 'Product Launch', '2026-03-11', 100, 6000.00,
3000.00, 'Confirmed');
Query OK, 7 rows affected (0.01 sec)
```

Figure 7: Commands to insert BOOKING table's records

```
MariaDB [event]> SELECT * FROM BOOKING;
+------------+------------+---------+-----------+-------------------+------------
+------------+-------------+------------+-----------+
| BookingID | CustomerID | VenueID | PackageID | EventType         | EventDate
| GuestCount | TotalAmount | BalanceDue | Status    |
+------------+------------+---------+-----------+-------------------+------------
+------------+-------------+------------+-----------+
|          1 |       1000 |    2000 |      3002 | Wedding           | 2026-03-06
|        350 |    11000.00 |    3000.00 | Confirmed |
|          2 |       1001 |    2002 |      3003 | Corporate Meeting | 2026-02-19
|         80 |     5000.00 |       0.00 | Paid      |
|          3 |       1002 |    2001 |      3000 | Birthday Party    | 2026-03-21
|        120 |     4300.00 |    2150.00 | Confirmed |
|          4 |       1003 |    2000 |      3004 | Wedding           | 2026-04-05
|        450 |    13000.00 |    6500.00 | Pending   |
|          5 |       1004 |    2003 |      3001 | Anniversary       | 2026-02-24
|         75 |     6700.00 |       0.00 | Paid      |
|          6 |       1000 |    2004 |      3003 | Training Workshop | 2026-02-14
|         40 |     4300.00 |    1000.00 | Confirmed |
|          7 |       1002 |    2002 |      3001 | Product Launch    | 2026-03-11
|        100 |     6000.00 |    3000.00 | Confirmed |
+------------+------------+---------+-----------+-------------------+------------
+------------+-------------+------------+-----------+
```

Figure 8: BOOKING table with records

4.2.2.2 Update Record

1) With Primary Key

```
MariaDB [event]> UPDATE BOOKING
    -> SET Status = 'Paid'
    -> WHERE BookingID = 4;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Figure 1: Update Record with Primary Key

Figure 1 shows the command to update the booking status using the primary key (BookingID). BookingID 4 was updated from 'Pending' to 'Paid' in the BOOKING table.

2) With Foreign Key

```
MariaDB [event]> UPDATE BOOKING
    -> SET BalanceDue = 0.00, Status = 'Paid'
    -> WHERE CustomerID = 1000;
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

Figure 2: Update Record with Foreign Key

Figure 2 shows the command to update booking records using a foreign key (CustomerID). All bookings belonging to CustomerID 1000 (Ahmad Hafiz) had their BalanceDue set to 0.00 and Status updated to 'Paid'.

4.2.2.3 Queries and Joins

1) For Single Table

- CUSTOMER table

```
MariaDB [event]> SELECT * FROM CUSTOMER;
+------------+--------------------+--------------------------+-------------+----
----------------------------+
| CustomerID | CusName            | CusEmail                 | CusPhone    |
CusAddress                    |
+------------+--------------------+--------------------------+-------------+----
----------------------------+
|       1000 | Ahmad Hafiz        | ahmad.hafiz@email.com    | 013-4567890 |
Jalan Tun Razak, Kuantan      |
|       1001 | Siti Nurhaliza     | siti.nur@email.com       | 019-2345678 |
Bandar Indera Mahkota, Kuantan |
|       1002 | Elizabeth Tan      | lizzytan@email.com       | 012-7654321 |
Inderapura, Kuantan           |
|       1003 | Mohd Azwan Md Nor  | wakdoyok@email.com       | 011-8765432 |
Jalan IM3, Kuantan            |
|       1004 | Mohd Syafiq Shamim | syafiqkyle@email.com     | 014-3456789 |
Kotasas, Kuantan              |
+------------+--------------------+--------------------------+-------------+----
----------------------------+
5 rows in set (0.00 sec)
```

Figure 1: Retrieving all customer details

Figure 1 shows the command to retrieve all customer information from the CUSTOMER table. This query is useful for displaying the complete customer registry of the resort.

- BOOKING table (Confirmed bookings)

```
MariaDB [event]> SELECT BookingID, EventType, EventDate, GuestCount, Status
    -> FROM BOOKING
    -> WHERE Status = 'Confirmed';
+-----------+------------------+------------+------------+-----------+
| BookingID | EventType        | EventDate  | GuestCount | Status    |
+-----------+------------------+------------+------------+-----------+
|         1 | Wedding          | 2026-03-06 |        350 | Confirmed |
|         3 | Birthday Party   | 2026-03-21 |        120 | Confirmed |
|         6 | Training Workshop | 2026-02-14 |         40 | Confirmed |
|         7 | Product Launch   | 2026-03-11 |        100 | Confirmed |
+-----------+------------------+------------+------------+-----------+
4 rows in set (0.00 sec)
```

Figure 2: Retrieving confirmed bookings from BOOKING table

Figure 2 shows the query to retrieve all confirmed bookings from the BOOKING table. This is important for the resort's operations team to identify bookings that have been verified but not yet fully paid.

2) For Joins Table

- BOOKING table + CUSTOMER table

```
MariaDB [event]> SELECT B.BookingID, C.CusName, C.CusPhone,
    -> B.EventType, B.EventDate, B.Status
    -> FROM BOOKING B
    -> LEFT JOIN CUSTOMER C ON B.CustomerID = C.CustomerID;
+-----------+-------------------+-------------+------------------+------------
+-----------+
| BookingID | CusName           | CusPhone    | EventType        | EventDate
| Status    |
+-----------+-------------------+-------------+------------------+------------
+-----------+
|         1 | Ahmad Hafiz       | 013-4567890 | Wedding          | 2026-03-06
| Confirmed |
|         2 | Siti Nurhaliza    | 019-2345678 | Corporate Meeting | 2026-02-19
| Paid      |
|         3 | Elizabeth Tan     | 012-7654321 | Birthday Party   | 2026-03-21
| Confirmed |
|         4 | Mohd Azwan Md Nor | 011-8765432 | Wedding          | 2026-04-05
| Paid      |
|         5 | Mohd Syafiq Shamim | 014-3456789 | Anniversary      | 2026-02-24
| Paid      |
|         6 | Ahmad Hafiz       | 013-4567890 | Training Workshop | 2026-02-14
| Confirmed |
|         7 | Elizabeth Tan     | 012-7654321 | Product Launch   | 2026-03-11
| Confirmed |
+-----------+-------------------+-------------+------------------+------------
+-----------+
```

Figure 3: Joining BOOKING table and CUSTOMER table (LEFT JOIN)

Figure 3 shows a LEFT JOIN between the BOOKING and CUSTOMER tables to retrieve booking details alongside the respective customer's name and phone number. This query is useful for generating booking reports with customer contact information.

- BOOKING table + VENUE table + PACKAGE table

```
MariaDB [event]> SELECT B.BookingID, B.EventType, V.VenueName,
    -> P.PackName, B.TotalAmount, B.Status
    -> FROM BOOKING B
    -> INNER JOIN VENUE V ON B.VenueID = V.VenueID
    -> INNER JOIN PACKAGE P ON B.PackageID = P.PackageID;
+-----------+------------------+------------------+-----------------+--------
-----+-----------+
| BookingID | EventType        | VenueName        | PackName        |
TotalAmount | Status    |
+-----------+------------------+------------------+-----------------+--------
-----+-----------+
|         1 | Wedding          | Grand Ballroom   | Premium Package |
11000.00 | Confirmed |
|         2 | Corporate Meeting | Executive Hall   | Corporate Package|
5000.00 | Paid      |
|         3 | Birthday Party   | Garden Pavilion  | Basic Package   |
4300.00 | Confirmed |
|         4 | Wedding          | Grand Ballroom   | Wedding Package |
13000.00 | Paid      |
|         5 | Anniversary      | Rooftop Terrace  | Normal Package  |
6700.00 | Paid      |
|         6 | Training Workshop | Conference Room A | Corporate Package|
4300.00 | Confirmed |
|         7 | Product Launch   | Executive Hall   | Normal Package  |
6000.00 | Confirmed |
+-----------+------------------+------------------+-----------------+--------
-----+-----------+
```

Figure 4: Joining BOOKING, VENUE, and PACKAGE tables (INNER JOIN)

Figure 4 shows an INNER JOIN across BOOKING, VENUE, and PACKAGE tables. This query retrieves a comprehensive booking summary showing the venue name, selected package, total amount, and current status. This is valuable for management reporting and financial reconciliation.

DATABASE ADMINISTRATION DOCUMENTATION

5.1 Security

Database Host: localhost

Port: 3306

Username	Roles
root	Admin role that has all privileges as a default user in MySQL
event_manager	User role that has limited privileges for general operations such as SELECT, INSERT, UPDATE, and DELETE on the event database.

User account 'event_manager' is used for all regular database operations instead of 'root' in order to prevent accidental or unauthorized changes to the database structure.

5.2 Backup and Restore

Backup Policy

- Backup Frequency: Daily at midnight
- Type of Backup: Incremental backups every 6 hours

Incremental backups are chosen because they take less time compared to full backups, resulting in better backup speed. The retention period is set to 7 days after each backup to free up storage space.

Backup Location: `C:\xampp\mysql\backup`

Backup Command:

```
C:\xampp>
# mysqldump -u root -p event > C:\xampp\mysql\backup\event.sql
Enter password:
```

Figure 1: Backup command

Figure 1 shows the backup command to backup the event database. The admin account 'root' is used in order to gain full privileges to perform the backup.

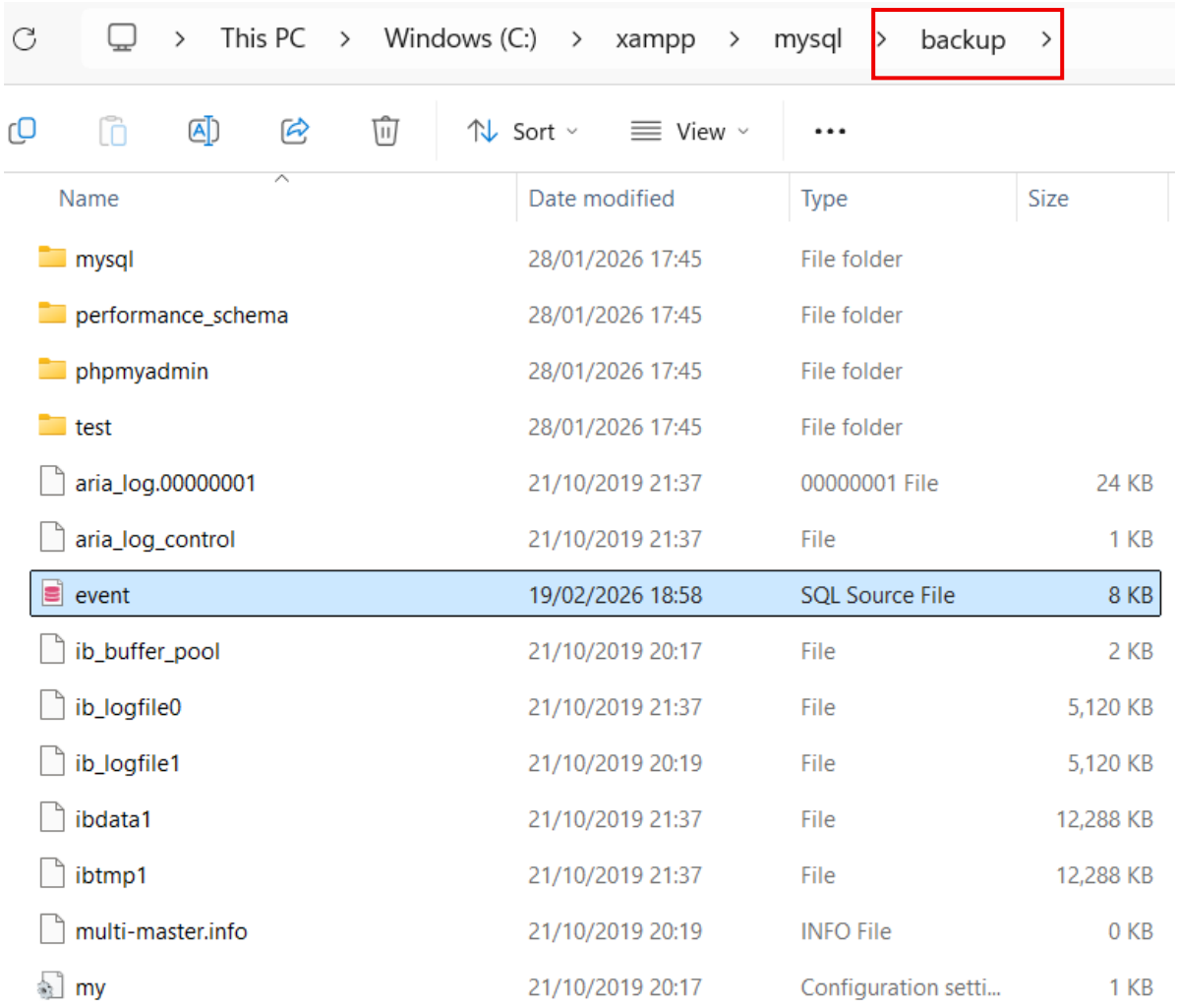

Figure 2: Backup database file in backup location

Restore Command:

```
C:\xampp>
# mysql -u root -p event < C:\xampp\mysql\backup\event.sql
Enter password:
```

Figure 3: Restore command

5.3 Basic Maintenance

Maintenance

Type of Maintenance	Period
Optimize tables	Weekly
Check table corruptions or errors	Weekly
Repair tables	If required
Checks and repair after shutdowns or crashes	After unexpected server shutdowns or crashes

Optimize tables commands:

```
MariaDB [event]> OPTIMIZE TABLE customer, venue, package, booking;
+-------------------------------------+----------+----------+-------------------
-------------------------------------------------+
| Table                               | Op       | Msg_type | Msg_text
|
+-------------------------------------+----------+----------+-------------------
-------------------------------------------------+
| event.customer       | optimize | note     | Table does not support optimize,
doing recreate + analyze instead |
| event.customer       | optimize | status   | OK
|
| event.venue          | optimize | note     | Table does not support optimize,
doing recreate + analyze instead |
| event.venue          | optimize | status   | OK
|
| event.package        | optimize | note     | Table does not support optimize,
doing recreate + analyze instead |
| event.package        | optimize | status   | OK
|
| event.booking        | optimize | note     | Table does not support optimize,
doing recreate + analyze instead |
| event.booking        | optimize | status   | OK
|
+-------------------------------------+----------+----------+-------------------
-------------------------------------------------+
```

Figure 3: Optimize tables commands

Check table corruptions or errors commands:

```
MariaDB [event]> CHECK TABLE customer, venue, package, booking;
+-------------------------------------+-------+----------+----------+
| Table                               | Op    | Msg_type | Msg_text |
+-------------------------------------+-------+----------+----------+
| event.customer       | check | status   | OK       |
| event.venue          | check | status   | OK       |
| event.package        | check | status   | OK       |
| event.booking        | check | status   | OK       |
+-------------------------------------+-------+----------+----------+
4 rows in set (0.00 sec)
```

Figure 4: Check table commands

Repair tables commands:

```
MariaDB [event]> REPAIR TABLE customer, venue, package, booking;
+-------------------------------------+--------+----------+---------------------
-------------------------------------+
| Table                               | Op     | Msg_type | Msg_text
|
+-------------------------------------+--------+----------+---------------------
-------------------------------------+
| event.customer       | repair | note     | The storage engine for the table
doesn't support repair  |
| event.venue          | repair | note     | The storage engine for the table
doesn't support repair  |
| event.package        | repair | note     | The storage engine for the table
doesn't support repair  |
| event.booking        | repair | note     | The storage engine for the table
doesn't support repair  |
+-------------------------------------+--------+----------+---------------------
-------------------------------------+
```

Figure 5: Repair tables commands

Checks and repair after shutdowns or crashes:

To repair a crashed table, the administrator is required to first check the tables for errors using the CHECK TABLE command, and then repair the errors using the REPAIR TABLE command, as demonstrated in Figures 4 and 5 above.

REFERENCES

event database:

1. http://localhost/phpmyadmin/db_structure.php?server=1&db=event
2. GeeksforGeeks. (2019, August 26). SQL | DDL, DQL, DML, DCL and TCL Commands. GeeksforGeeks. https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/
3. Upadhyay, N. (2020, May 12). How to backup and restore MySQL databases using the mysqldump command. SQL Shack. https://www.sqlshack.com/how-to-backup-and-restore-mysql-databases-using-the-mysqldump-command/
4. Amata, J. (n.d.). How to Repair a Corrupted MySQL Table. Docs.vultr.com. https://docs.vultr.com/how-to-repair-a-corrupted-mysql-table
5. How to Repair Crashed Tables With phpMyAdmin. (2023, May 8). Nexcess. https://www.nexcess.net/help/how-to-repair-mysql-tables/

APPENDICES

7.1 Assessment Rubric

(Rubric as per provided in the Final Project guidelines for CSC 1393 – Database Fundamentals, Session 3 2025/2026)